

实战：LangChain 版 OpenAI-Translator v2.0

彭靖田

Google Developer Experts
LangChain Developer



目录

- 深入理解 Chat Model 和 Chat Prompt Template
 - 温故：LangChain Chat Model 使用方法和流程
 - 使用 Chat Prompt Template 设计翻译提示模板
 - 使用 Chat Model 实现双语翻译
 - 使用 LLMChain 简化构造 Chat Prompt
- 基于 LangChain 优化 OpenAI-Translator 架构设计
 - 由 LangChain 框架接手大模型管理
 - 聚焦应用自身的 Prompt 设计
 - 使用 TranslationChain 实现翻译接口
 - 更简洁统一的配置管理
- OpenAI-Translator v2.0 功能特性研发
 - 基于Gradio的图形化界面设计与实现
 - 基于 Flask 的 Web Server 设计与实现

深入理解

Chat Model 和 Chat Prompt Template


```
from langchain.chat_models import ChatOpenAI
chat_model = ChatOpenAI(model_name="gpt-3.5-turbo")

from langchain.schema import (
    AIMessage,
    HumanMessage,
    SystemMessage
)

messages = [SystemMessage(content="You are a helpful assistant."),
             HumanMessage(content="Who won the world series in 2020?"),
             AIMessage(content="The Los Angeles Dodgers won the World Series in 2020."),
             HumanMessage(content="Where was it played?")]

print(messages)

chat_model(messages)
```

调用 chat_model(messages) 返回结果:

```
AIMessage(
  content="The 2020 World Series was played at Globe Life Field in Arlington, Texas.",
  additional_kwargs={},
  example=False,
)
```


使用 Chat Prompt Template 设计翻译提示模板

```
from langchain.schema import AIMessage, HumanMessage, SystemMessage
# 导入 Chat Model 即将使用的 Prompt Templates
from langchain.prompts.chat import (
    ChatPromptTemplate,
    SystemMessagePromptTemplate,
    AIMessagePromptTemplate,
    HumanMessagePromptTemplate,
)

# 翻译任务指令始终由 System 角色承担
template = (
    """You are a translation expert, proficient in various languages. \n
    Translates English to Chinese."""
)
system_message_prompt = SystemMessagePromptTemplate.from_template(template)

print(system_message_prompt)

prompt=PromptTemplate(input_variables=[], output_parser=None, partial_variables={},
template='You are a translation expert, proficient in various languages. \n\n Tr
anslates English to Chinese.', template_format='f-string', validate_template=True)
additional_kwargs={}

# 待翻译文本由 Human 角色输入
human_template = "{text}"
human_message_prompt = HumanMessagePromptTemplate.from_template(human_template)

print(human_message_prompt)

HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['text'], output_p
arser=None, partial_variables={}, template='{text}', template_format='f-string', va
lidate_template=True), additional_kwargs={})
```



```
# 使用 System 和 Human 角色的提示模板构造 ChatPromptTemplate
chat_prompt_template = ChatPromptTemplate.from_messages(
    [system_message_prompt, human_message_prompt]
)
```

```
print(chat_prompt_template)
```

```
input_variables=['text'] output_parser=None partial_variables={} messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], output_parser=None, partial_variables={}, template='You are a translation expert, proficient in various languages. \n\n Translates English to Chinese.', template_format='f-string', validate_template=True), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['text'], output_parser=None, partial_variables={}, template='{text}', template_format='f-string', validate_template=True), additional_kwargs={})]
```

```
### 规范化 Python 复杂对象...
```

```
# 生成用于翻译的 Chat Prompt
```

```
chat_prompt_template.format_prompt(text="I love programming.")
```

```
ChatPromptValue(messages=[SystemMessage(content='You are a translation expert, proficient in various languages. \n\n Translates English to Chinese.', additional_kwargs={}), HumanMessage(content='I love programming.', additional_kwargs={}, example=False)])
```



```
# 使用 System 和 Human 角色的提示模板构造 ChatPromptTemplate
chat_prompt_template = ChatPromptTemplate.from_messages(
    [system_message_prompt, human_message_prompt]
)
```

```
print(chat_prompt_template)
```

```
input_variables=['text'] output_parser=None partial_variables={} messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], output_parser=None, partial_variables={}, template='You are a translation expert, proficient in various languages. \n\n Translates English to Chinese.', template_format='f-string', validate_template=True), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['text'], output_parser=None, partial_variables={}, template='{text}', template_format='f-string', validate_template=True), additional_kwargs={})]
```

```
### 规范化 Python 复杂对象...
```

```
# 生成用于翻译的 Chat Prompt
```

```
chat_prompt_template.format_prompt(text="I love programming.")
```

```
ChatPromptValue(messages=[SystemMessage(content='You are a translation expert, proficient in various languages. \n\n Translates English to Chinese.', additional_kwargs={}), HumanMessage(content='I love programming.', additional_kwargs={}, example=False)])
```

规范化 Python 复杂对象

Sample

1 messages=[SystemMessagePromptTemplate(prompt=PromptTemplate(input_variables=[], output_parser=None, partial_variables={}, template='You are a translation expert, proficient in various languages. \n\n Translates English to Chinese.', template_format='f-string', validate_template=True), additional_kwargs={}), HumanMessagePromptTemplate(prompt=PromptTemplate(input_variables=['text'], output_parser=None, partial_variables={}, template='{text}', template_format='f-string', validate_template=True), additional_kwargs={})]

Ln: 1 Col: 523 size: 523 B

File URL

Python Version

Format

Advertisements

Download

Output

1 messages = [
2 SystemMessagePromptTemplate(
3 prompt=PromptTemplate(
4 input_variables=[],
5 output_parser=None,
6 partial_variables={},
7 template="You are a translation expert,
8 proficient in various languages. \n\n
9 Translates English to Chinese.",
10 template_format="f-string",
11 validate_template=True,
12),
13 additional_kwargs={},
14),
15 HumanMessagePromptTemplate(
16 prompt=PromptTemplate(
17 input_variables=["text"],
18 output_parser=None,
19 partial_variables={},
20 template="{text}",
21 template_format="f-string",
22 validate_template=True,
23),
24 additional_kwargs={},
25),
26]

Ln: 25 Col: 0 size: 772 B

使用 Chat Model 实现双语翻译

使用 chat_prompt_template.to_messages 方法生成 Messages

生成聊天模型真正可用的消息记录 Messages

```
chat_prompt = chat_prompt_template.format_prompt(text="I love programming.").to_messages()
```

```
chat_prompt
```

```
[SystemMessage(content='You are a translation expert, proficient in various languages. \n\n    Translates English to Chinese.', additional_kwargs={}),  
 HumanMessage(content='I love programming.', additional_kwargs={}, example=False)]
```

使用 Chat Model (GPT-3.5-turbo) 实际执行翻译任务

```
from langchain.chat_models import ChatOpenAI
```

为了翻译结果的稳定性, 将 temperature 设置为 0

```
translation_model = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)
```

```
translation_result = translation_model(chat_prompt)
```

查看翻译结果

```
print(translation_result.content)
```

我喜欢编程。

使用 LLMChain 简化构造 ChatPrompt

```
from langchain.chains import LLMChain
```

```
# 无需再每次都使用 to_messages 方法构造 Chat Prompt
```

```
translation_chain = LLMChain(llm=translation_model, prompt=chat_prompt_template)
```

```
# 等价于 translation_result.content (字符串类型)
```

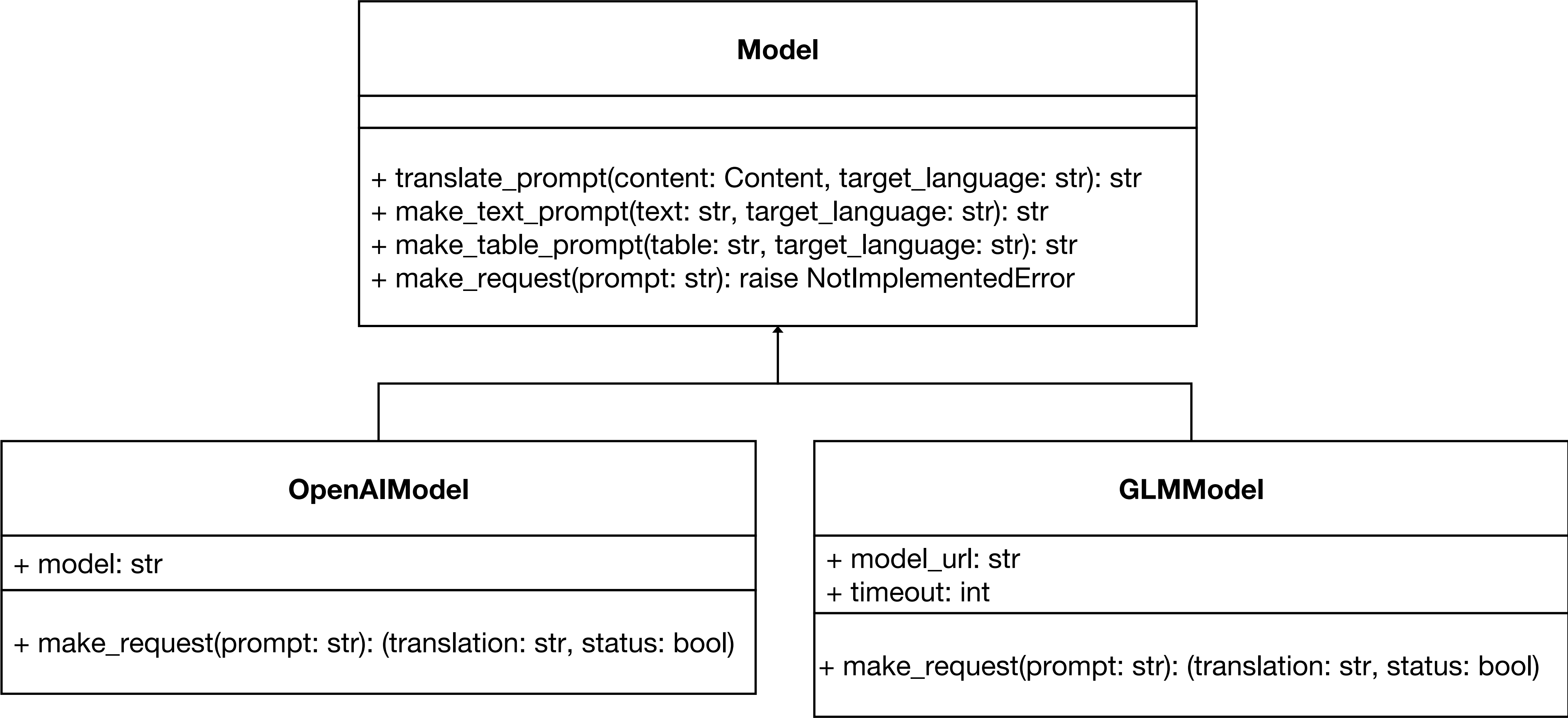
```
chain_result = translation_chain.run({'text': "I love programming."})
```

```
print(chain_result)
```

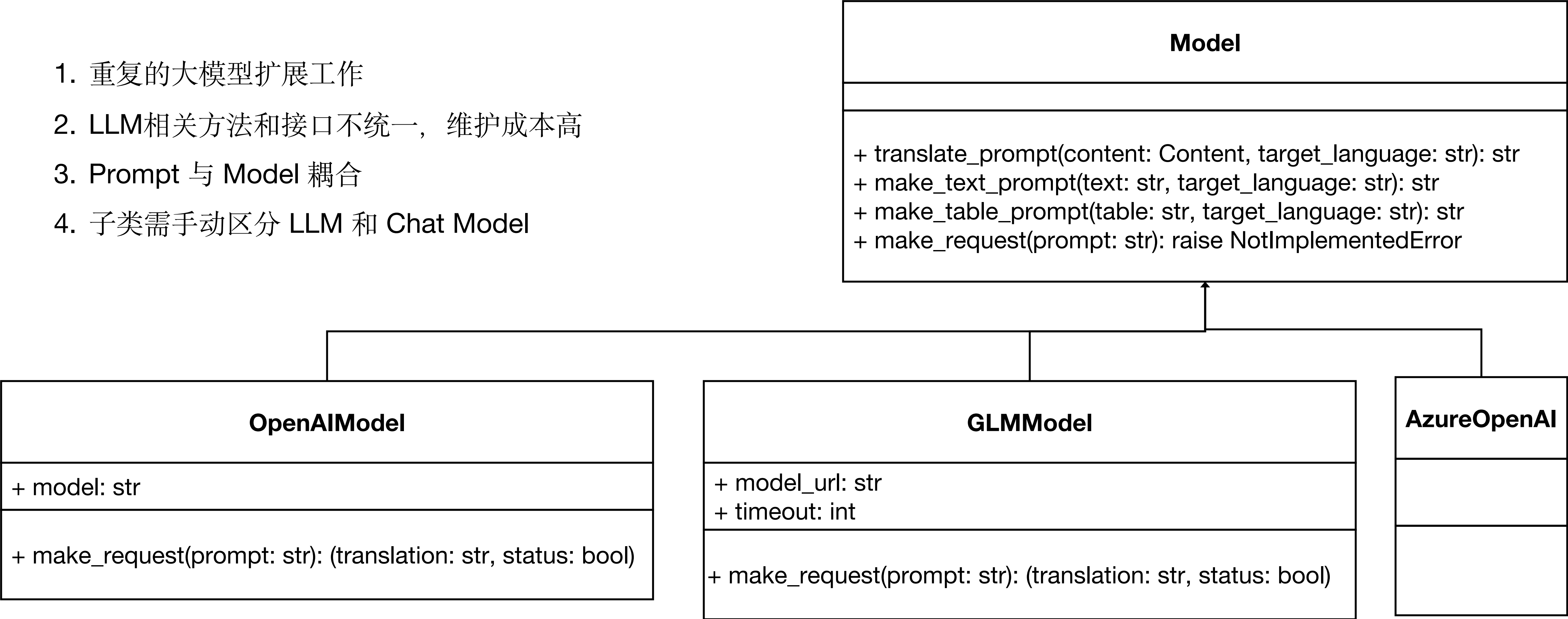
我喜欢编程。

DEMO

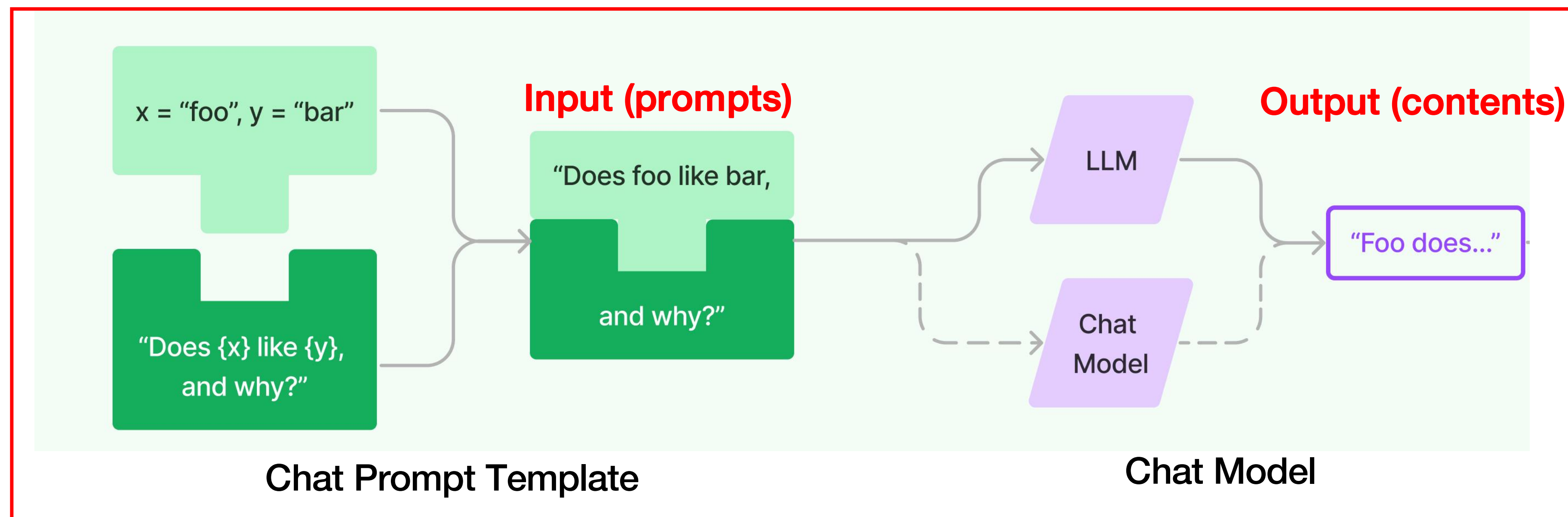
基于 **LangChain** 优化 **OpenAI-Translator** 架构设计



- 1. 重复的大模型扩展工作
- 2. LLM相关方法和接口不统一， 维护成本高
- 3. Prompt 与 Model 耦合
- 4. 子类需手动区分 LLM 和 Chat Model



TranslationChain (LLMChain)



1. 重复的大模型扩展工作
2. LLM相关方法和接口不统一, 维护成本高
3. Prompt 与 Model 耦合
4. 子类需手动区分 LLM 和 Chat Model



1. 使用 LangChain 框架替代自己造轮子
2. Model I/O 覆盖主流大模型, 提供标准接口
3. 使用 Chains 管理 Prompt 与 Model
4. 框架原生支持 LLM 和 Chat Model 两类模型

Translation Chain

```
class TranslationChain:
    def __init__(self, model_name: str = "gpt-3.5-turbo", verbose: bool = True):
        # 翻译任务指令始终由 System 角色承担
        template = (
            """You are a translation expert, proficient in various languages. \n
            Translates {source_language} to {target_language}."""
        )
        system_message_prompt = SystemMessagePromptTemplate.from_template(template)

        # 待翻译文本由 Human 角色输入
        human_template = "{text}"
        human_message_prompt = HumanMessagePromptTemplate.from_template(human_template)

        # 使用 System 和 Human 角色的提示模板构造 ChatPromptTemplate
        chat_prompt_template = ChatPromptTemplate.from_messages(
            [system_message_prompt, human_message_prompt]
        )

        # 为了翻译结果的稳定性, 将 temperature 设置为 0
        chat = ChatOpenAI(model_name=model_name, temperature=0, verbose=verbose)

        self.chain = LLMChain(llm=chat, prompt=chat_prompt_template, verbose=verbose)

    def run(self, text: str, source_language: str, target_language: str) -> (str, bool):
        result = ""
        try:
            result = self.chain.run({
                "text": text,
                "source_language": source_language,
                "target_language": target_language,
            })
        except Exception as e:
            LOG.error(f"An error occurred during translation: {e}")
            return result, False

        return result, True
```


解耦

Model 与 Prompt

```
class PDFTranslator:
```

```
    def __init__(self, model_name: str):
        self.translate_chain = TranslationChain(model_name)
        self.pdf_parser = PDFParser()
        self.writer = Writer()
```

```
    def translate_pdf(self,
                      input_file: str,
                      output_file_format: str = 'markdown',
                      source_language: str = "English",
                      target_language: str = 'Chinese',
                      pages: Optional[int] = None):
```

```
        self.book = self.pdf_parser.parse_pdf(input_file, pages)
```

```
        for page_idx, page in enumerate(self.book.pages):
            for content_idx, content in enumerate(page.contents):
```

```
                # Translate content.original
```

```
                translation, status = self.translate_chain.run(content, source_language, target_language)
```

```
                # Update the content in self.book.pages directly
```

```
                self.book.pages[page_idx].contents[content_idx].set_translation(translation, status)
```

```
        return self.writer.save_translated_book(self.book, output_file_format)
```

```
        for page_idx, page in enumerate(self.book.pages):
            for content_idx, content in enumerate(page.contents):
                prompt = self.model.translate_prompt(content, target_language)
                translation, status = self.model.make_request(prompt)
```


单例模式 全局配置管理

```
import sys
import os

sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from utils import ArgumentParser, LOG
from translator import PDFTranslator, TranslationConfig

if __name__ == "__main__":
    # 解析命令行
    argument_parser = ArgumentParser()
    args = argument_parser.parse_arguments()

    # 初始化配置单例
    config = TranslationConfig()
    config.initialize(args)

    # 实例化 PDFTranslator 类, 并调用 translate_pdf() 方法
    translator = PDFTranslator(config.model_name)
    translator.translate_pdf(config.input_file, config.output_file_format, pages=None)
```

单例模式 全局配置管理

```
class TranslationConfig:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(TranslationConfig, cls).__new__(cls)
            cls._instance._config = None
        return cls._instance

    def initialize(self, args):
        with open(args.config_file, "r") as f:
            config = yaml.safe_load(f)

        # Use the argparse Namespace to update the configuration
        overridden_values = {
            key: value for key, value in vars(args).items() if key in config and value is not None
        }
        config.update(overridden_values)

        # Store the original config dictionary
        self._instance._config = config

    def __getattr__(self, name):
        # Try to get attribute from _config
        if self._instance._config and name in self._instance._config:
            return self._instance._config[name]
        raise AttributeError(f"'TranslationConfig' object has no attribute '{name}'")
```


DEMO

OpenAI-Translator v2.0 功能特性研发

Build & Share Delightful Machine Learning Apps

Gradio is the fastest way to demo your machine learning model with a friendly web interface so that anyone can use it, anywhere!

[Get Started](#)
 **Star**
21141
[Sketch Recognition](#)
[Stable Diffusion](#)
[Time Series Forecasting](#)
[XGBoost with Explainability](#)
[Chat with Llama 2](#)

OpenAI-Translator v2.0 (PDF 电子书翻译工具)

上传PDF文件

Drop File Here
- or -
Click to Upload

下载翻译文件

源语言 (默认: 英文)

English

目标语言 (默认: 中文)

Chinese

Clear

Submit

Gradio GUI

```
def translation(input_file, source_language, target_language):
    LOG.debug(f"[翻译任务]\n源文件: {input_file.name}\n源语言: {source_language}\n目标语言: {target_language}")

    output_file_path = Translator.translate_pdf(
        input_file.name, source_language=source_language, target_language=target_language)

    return output_file_path

def launch_gradio():

    iface = gr.Interface(
        fn=translation,
        title="OpenAI-Translator v2.0(PDF 电子书翻译工具)",
        inputs=[
            gr.File(label="上传PDF文件"),
            gr.Textbox(label="源语言 (默认: 英文) ", placeholder="English", value="English"),
            gr.Textbox(label="目标语言 (默认: 中文) ", placeholder="Chinese", value="Chinese")
        ],
        outputs=[
            gr.File(label="下载翻译文件")
        ],
        allow_flagging="never"
    )

    iface.launch(share=True, server_name="0.0.0.0")
```

REST API using Flask



+



Flask

Flask Web Server

```
TEMP_FILE_DIR = "flask_temps/"

@app.route('/translation', methods=['POST'])
def translation():
    try:
        input_file = request.files['input_file']
        source_language = request.form.get('source_language', 'English')
        target_language = request.form.get('target_language', 'Chinese')

        LOG.debug(f"[input_file]\n{input_file}")
        LOG.debug(f"[input_file.filename]\n{input_file.filename}")

        if input_file and input_file.filename:
            # # 创建临时文件
            input_file_path = TEMP_FILE_DIR+input_file.filename
            LOG.debug(f"[input_file_path]\n{input_file_path}")

            input_file.save(input_file_path)

            # 调用翻译函数
            output_file_path = Translator.translate_pdf(
                input_file=input_file_path,
                source_language=source_language,
                target_language=target_language)

            # 移除临时文件
            # os.remove(input_file_path)

            # 构造完整的文件路径
            output_file_path = os.getcwd() + "/" + output_file_path
            LOG.debug(output_file_path)

            # 返回翻译后的文件
            return send_file(output_file_path, as_attachment=True)

    except Exception as e:
        response = {
            'status': 'error',
            'message': str(e)
        }
        return jsonify(response), 400
```

API 调用 翻译服务

使用 requests 库请求翻译服务 API

```
import requests

# Flask服务器的地址
FLASK_SERVER_URL = 'http://localhost:5000'

# 翻译服务接口
translation_url = f'{FLASK_SERVER_URL}/translation'

# 要上传的文件路径
file_path = '../tests/test.pdf' # 修改为你的文件路径

# 构建请求参数
params = {
    'source_language': 'English', # 修改为你的源语言
    'target_language': 'Chinese' # 修改为你的目标语言
}

# 发送POST请求
with open(file_path, 'rb') as file:
    files = {'input_file': file}
    response = requests.post(translation_url, files=files, data=params)

# 翻译后文件
output_filename = "translated_output.md"

# 处理响应
if response.status_code == 200:
    # 保存翻译后的文件
    with open(output_filename, 'wb') as output_file:
        output_file.write(response.content)
    print(f"Translation completed. Translated file saved as {output_filename}.")
else:
    print(f"Translation failed. Status code: {response.status_code}")
    print(response.text)
```

Translation completed. Translated file saved as translated_output.md.

翻译服务端 日志输出

```
127.0.0.1 -- [22/Aug/2023 18:23:45] "POST /translation HTTP/1.1" 200 -
2023-08-22 18:24:09.650 | DEBUG      | __main__:translation:21 - [input_file]
<FileStorage: 'test.pdf' (None)>
2023-08-22 18:24:09.650 | DEBUG      | __main__:translation:22 - [input_file.filename]
test.pdf
2023-08-22 18:24:09.651 | DEBUG      | __main__:translation:27 - [input_file_path]
flask_temps/test.pdf
2023-08-22 18:24:09.740 | DEBUG      | translator.pdf_parser:parse_pdf:46 - [raw_text]
Test Data
This dataset contains two test samples provided by ChatGPT, an AI language model by OpenAI.
These samples include a markdown table and an English text passage, which can be used to test an
English-to-Chinese translation software supporting both text and table formats.
Text testing
The quick brown fox jumps over the lazy dog. This pangram contains every letter of the English
alphabet at least once. Pangrams are often used to test fonts, keyboards, and other text-related
tools. In addition to English, there are pangrams in many other languages. Some pangrams are more
difficult to construct due to the unique characteristics of the language.
Table Testing
2023-08-22 18:24:09.743 | DEBUG      | translator.pdf_parser:parse_pdf:54 - [table]
[Fruit, Color, Price (USD)] [Apple, Red, 1.20] [Banana, Yellow, 0.50] [Orange, Orange, 0.80] [Strawberry, Red, 2.50] [Blueberry, Blue, 3.00] [Kiwi, Green, 1.00] [Mango, Orange, 1.50] [Grape, Purple, 2.00]

> Entering new LLMChain chain...
Prompt after formatting:
System: You are a translation expert, proficient in various languages.

Translates English to Chinese.
```

课程项目：GitHub openai-quickstart

openai-quickstartPublic

UnpinUnwatch 11Fork 171Starred 351

main1 branch0 tags

Go to fileAdd fileCode

DjangoPeng add langchain-refactored version of openai-translator1f66fba 17 hours ago67 commits

chatgpt-plugins	add initial version of chatgpt plugins	3 weeks ago
docs	make layout more readable	last month
langchain	add langchain-refactored version of openai-translator	17 hours ago
openai-translator	Merge branch 'main' of github.com:DjangoPeng/openai-quickstart in...	3 weeks ago
openai_api	Remove fine_food_reviews_with_embeddings_1k_demo.csv	last month
.gitignore	add initial version of agents and data	last week
LICENSE	Initial commit	last month
README-CN.md	docs: add 8/13 schedule to readme (#49)	4 days ago
README.md	docs: add 8/13 schedule to readme (#49)	4 days ago

README.md

OpenAI Quickstart

English | 中文

This project is designed as a one-stop learning resource for anyone interested in large language models and their application in Artificial Intelligence Governance and Control (AIGC) scenarios. By providing theoretical foundations, development basics, and hands-on examples, this project offers comprehensive guidance on these cutting-edge topics.

About

A comprehensive guide to understanding and implementing large language models with hands-on examples using LangChain for AIGC applications.

ReadmeApache-2.0 licenseActivity351 stars11 watching171 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 8